Smart Study planner

Project Report

SMART STUDY PLANNER

Academic Project Report

Submitted by: **Anushka**

Reg Number: **24BAI10805**

Course: **B.Tech**

Subject: **Programming in Java**

Session: **2026 – 2027**

Date: **25/03/2026**

VITyarthi - Build Your Own Project

1. Introduction

1.1 Project Overview

The Smart Study Planner is a Java console-based application designed to help students manage their daily study tasks in an organized way. It allows users to add, view, update, and delete tasks along with details like deadline and priority. The system also supports features like searching tasks, viewing pending tasks, and generating a summary of tasks. Additionally, it includes data persistence, which ensures that tasks are saved and remain available even after restarting the program.

1.2 Purpose

This project was developed as part of the course to apply key Java concepts such as object-oriented programming, collections (ArrayList), file handling, and user input handling.
It helped me understand how different concepts like data storage, logic handling, and user interaction can be combined to build a real-world application.

1.3 Scope

The application focuses on core task management functionalities including adding tasks, viewing them, marking them as completed, deleting tasks, searching tasks, and viewing task summaries.
It also implements file-based storage to maintain data across sessions.
Since this is a console-based project, the scope is limited to backend logic and functionality without any graphical user interface or database integration.

2. Problem Statement

2.1 Current Challenges

Students often face difficulty in managing their daily study tasks and deadlines. Some common issues are:

- Tasks are not organized properly
- Deadlines are forgotten or missed
- No clear way to track completed and pending work

• Difficulty in prioritizing important tasks
• No simple system to manage everything in one place

2.2 Impact

These problems can lead to:

• Poor time management
• Missed assignments or deadlines
• Increased stress and confusion
• Reduced productivity
• Difficulty in staying consistent with studies

2.3 Proposed Solution

To solve these issues, a Java-based console application is developed that allows users to manage their tasks in an organized way. The system provides features like adding tasks, viewing them, marking tasks as completed, deleting tasks, searching tasks, and viewing pending tasks.

It also includes a summary feature to track overall progress and file-based storage to ensure that tasks remain saved even after restarting the program. This helps in maintaining a structured and consistent workflow.

3. Functional Requirements

The system provides the following functionalities:

• The user can add a new task with subject, description, deadline, and priority
• The system stores multiple tasks using an ArrayList
• The user can view all added tasks in a structured format
• The user can mark any task as completed using its ID
• The user can delete a task using its ID
• Each task is assigned a unique ID automatically
• The system allows users to view only pending tasks
• The user can search tasks using keywords (subject or description)
• The system provides a summary showing total, completed, and pending tasks
• The system automatically saves tasks using file handling
• Saved tasks are loaded again when the program restarts (data persistence)

4. Non-Functional Requirements

• The system has a simple menu-driven interface which makes it easy to use
• The application gives quick response for normal operations
• User input is handled properly to avoid basic errors
• The code is divided into different classes to keep it clean and easy to understand
• The system can run on any system where Java is installed

5. System Design

The project follows a structured design by dividing the system into multiple components for better organization and clarity:

• **Task Class (Model):**
This class represents each task and stores details like subject, description, deadline, priority, and status. It also assigns a unique ID to every task.

• **TaskManager Class (Logic Layer):**
This class handles all the main operations such as adding, viewing, deleting, searching, and updating tasks. It manages the list of tasks and controls the overall functionality of the system.

• **FileHandler Class (Storage Layer):**
This class is responsible for saving and loading tasks using file handling. It ensures that tasks remain stored even after the program is restarted.

• **Main Class (SmartStudyPlanner):**
This class provides a menu-driven interface for user interaction. It takes input from the user and calls appropriate methods from the TaskManager class.

This separation of components helps in making the code more organized, readable, and easy to maintain.

6. Implementation Details

6.1 Task Class

The Task class is used to represent each individual task in the system. It stores details like subject, description, deadline, priority, and status. Each task is given a unique ID automatically.
I have used private variables with getters to maintain proper encapsulation. The class also includes methods to mark a task as completed and to check if it is still pending.

6.2 TaskManager Class

The TaskManager class handles all the main operations of the system. It manages a list of tasks using ArrayList.
All functionalities like adding a task, viewing tasks, marking a task as completed, deleting a task, searching tasks, and showing summary are implemented here.
This helps in keeping the logic separate from the user interface, making the code more organized and easier to manage.

6.3 FileHandler Class

The FileHandler class is responsible for saving and loading tasks using file handling.
I have used serialization to store the task data in a file so that the tasks are not lost when the program is closed.
Whenever a task is added, deleted, or updated, the data is saved automatically.
When the program starts again, the saved tasks are loaded back.

6.4 Main Class (SmartStudyPlanner)

This is the main class of the project which provides a menu-driven interface to the user.
It takes input from the user and calls the required methods from the TaskManager class based on the selected option.
This class is mainly responsible for handling user interaction and displaying outputs.

To demonstrate the working of the system, the following screenshots show different functionalities of the Smart Study Planner along with their outputs.

7.Output Screenshots

7.1 Main Menu

```
Welcome to Smart Study Planner!

==============================
     SMART STUDY PLANNER
==============================
1. Add Task
2. View Tasks
3. Mark Complete
4. Delete Task
5. View Pending Tasks
6. Search Task
7. Task Summary
8. Exit
Enter choice:
```

This shows the main interface of the system where different options are available to the user.

7.2 Adding a Task

```
Welcome to Smart Study Planner!

==============================
     SMART STUDY PLANNER
==============================
1. Add Task
2. View Tasks
3. Mark Complete
4. Delete Task
5. View Pending Tasks
6. Search Task
7. Task Summary
8. Exit
Enter choice: 1
Enter subject: Maths
Enter description: Tutorial
Enter deadline: 25-3-26
Priority (High/Medium/Low): High
Task added successfully.
```

This shows how a new task is added with subject, description, deadline, and priority.

7.3 Viewing All Tasks

```
==============================
     SMART STUDY PLANNER
==============================
1. Add Task
2. View Tasks
3. Mark Complete
4. Delete Task
5. View Pending Tasks
6. Search Task
7. Task Summary
8. Exit
Enter choice: 2

--- All Tasks ---
[ID: 1  ] Maths        | Tutorial                | Deadline: 25-3-26    |
Priority: High    | Status: Pending
```

Displays all the tasks stored in the system in a structured format.

7.4 Marking Task as Completed

```
==============================
     SMART STUDY PLANNER
==============================
1. Add Task
2. View Tasks
3. Mark Complete
4. Delete Task
5. View Pending Tasks
6. Search Task
7. Task Summary
8. Exit
Enter choice: 3

--- All Tasks ---
[ID: 1  ] Maths        | Tutorial                | Deadline: 25-3-26    |
Priority: High    | Status: Pending
Enter task ID: 1
Task marked as completed.
```

Shows how a task is marked as completed using its ID.

7.5 Deleting a Task

```
==============================
     SMART STUDY PLANNER
==============================
1. Add Task
2. View Tasks
3. Mark Complete
4. Delete Task
5. View Pending Tasks
6. Search Task
7. Task Summary
8. Exit
Enter choice: 4

--- All Tasks ---
[ID: 1  ] Maths        | Tutorial                  | Deadline: 25-3-26    |
Priority: High    | Status: Completed
[ID: 2  ] English      | Preparation for play      | Deadline: 3-4-26     |
Priority: Medium  | Status: Pending
Enter task ID: 1
Task deleted successfully.
```

Demonstrates deletion of a task from the system.

7.6 Viewing Pending Tasks

```
==============================
     SMART STUDY PLANNER
==============================
1. Add Task
2. View Tasks
3. Mark Complete
4. Delete Task
5. View Pending Tasks
6. Search Task
7. Task Summary
8. Exit
Enter choice: 5

--- Pending Tasks ---
[ID: 2  ] English      | Preparation for play      | Deadline: 3-4-26     |
Priority: Medium  | Status: Pending
```

Displays only the tasks that are still pending.

7.7 Search Functionality

```
==============================
     SMART STUDY PLANNER
==============================
1. Add Task
2. View Tasks
3. Mark Complete
4. Delete Task
5. View Pending Tasks
6. Search Task
7. Task Summary
8. Exit
Enter choice: 6
Enter keyword: Eng

--- Search Results ---
[ID: 2  ] English      | Preparation for play      | Deadline: 3-4-26     |
Priority: Medium  | Status: Pending
```

Shows how tasks can be searched using keywords.

7.8 Task Summary

```
==============================
     SMART STUDY PLANNER
==============================
1. Add Task
2. View Tasks
3. Mark Complete
4. Delete Task
5. View Pending Tasks
6. Search Task
7. Task Summary
8. Exit
Enter choice: 7

--- Task Summary ---
Total Tasks     : 1
Completed       : 0
Pending         : 1
```

Displays total, completed, and pending tasks.

7.9 Data Persistence

```
==============================
     SMART STUDY PLANNER
==============================
1. Add Task
2. View Tasks
3. Mark Complete
4. Delete Task
5. View Pending Tasks
6. Search Task
7. Task Summary
8. Exit
Enter choice: 2

--- All Tasks ---
[ID: 2  ] English      | Preparation for play      | Deadli
ne: 3-4-26     | Priority: Medium  | Status: Pending
```

This demonstrates that tasks are saved and loaded using file handling, ensuring data persistence across program runs.

8. Challenges Faced

While developing this project, I faced a few challenges:

• Understanding how to properly divide the program into different classes and maintain a clear structure
• Handling user input correctly using Scanner, especially managing nextInt() and nextLine() together
• Implementing a menu-driven system without logical errors
• Managing tasks using ArrayList and ensuring correct operations based on task ID
• Implementing file handling for saving and loading tasks, especially understanding serialization works.

9. Learning Outcomes

Through this project, I learned how to apply Java concepts in a practical way. I understood how classes and objects work together, how to store multiple data using ArrayList, and how to build a complete program with user interaction.

I also learned how to organize code using multiple classes, which made the program more structured and easier to manage.

Additionally, I gained an understanding of file handling using serialization, which helped in storing data permanently and loading it again when the program restarts.

Overall, this project helped me gain confidence in coding and gave me a clear idea of how real applications are designed and implemented.

10.Conclusion

The Smart Study Planner is a simple and effective application for managing daily study tasks. It helps in organizing tasks, tracking progress, and improving productivity.

The use of file handling ensures that data is not lost, making the system more reliable. The project also demonstrates the practical use of Java concepts like OOP, collections, and file handling.

Overall, this project provides a clear understanding of how a real-world application can be designed and implemented in a structured way.